



# SnapSpend Technical Report

Course: NLP in Industry

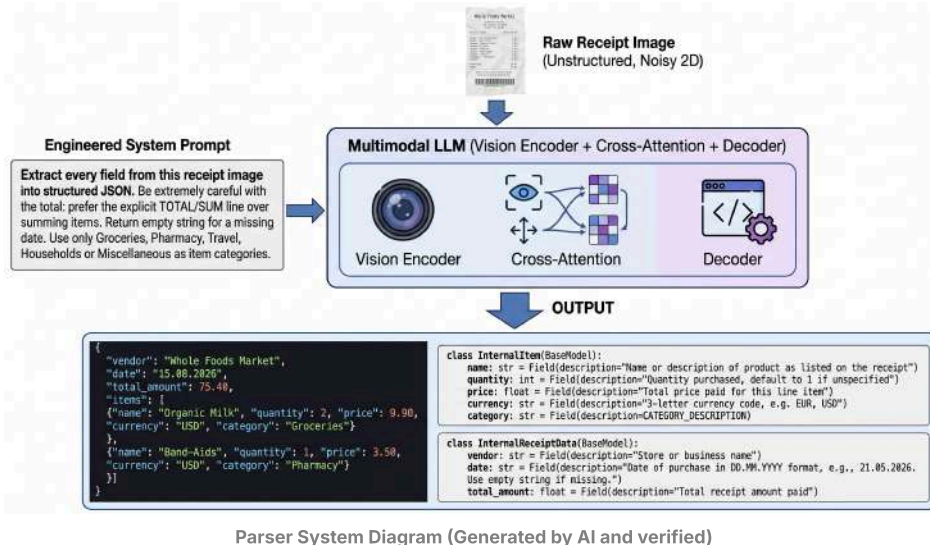
## 1. Abstract

Visual Information Extraction from unstructured physical documents (e.g., shopping receipts) remain a core challenge at the intersection of Computer Vision (CV) and Natural Language Processing (NLP). Traditional approaches typically use a two-stage pipeline: an Optical Character Recognition (OCR) engine extracts bounding-box-aligned text tokens, and a sequence-labeling or text-to-text NLP model then parses key-value pairs. These architectures are prone to compounding error propagation and disambiguation failures.

This report presents the technical details behind **SnapSpend**, a smart expense tracking system. It analyzes the shift from OCR-transcription pipelines to an end-to-end Multimodal Large Language Model approach that leverages **schema-constrained decoding** and **prompt engineering**. It also documents the prompt's iterative refinement—from a generic baseline to a schema-guaranteed formulation—introduces an evaluation framework with token-level and semantic metrics, and compares performance across multiple models/pipelines ( `gemini-3.7-flash` , `gemini-3.1-flash-lite` , `gemma-4-31b-it` , `gemma-4-26b-a4b-it` , and `gemini-3.1-flash-lite + PaddleOCR` ).

## 2. Problem Formulation & Task Definition

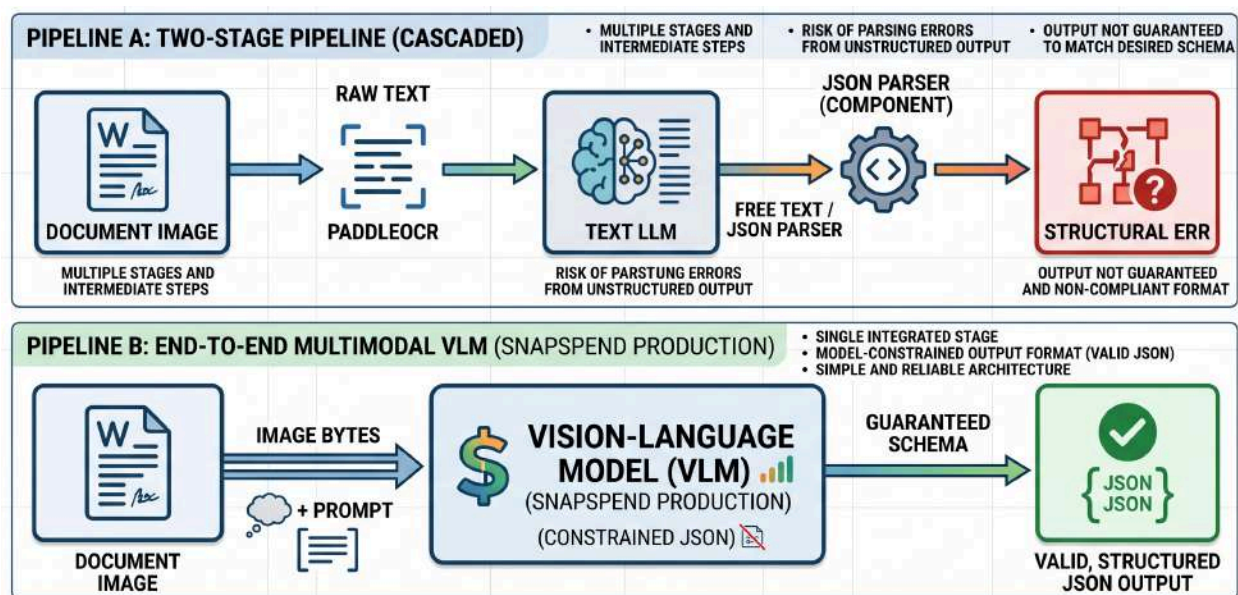
The core objective is **zero-shot/few-shot structured information extraction from receipt images**. Given a raw 2D receipt image, the system must emit a strictly typed JSON object:



## NLP Challenges in Document Understanding

- Spatial Layout & Reading Order:** Receipts often contain columnar/tabular layouts, multi-item line wraps, right-aligned monetary values, and variable subtotal/tax blocks.
- Arithmetic & Entity Disambiguation:** Models may confuse **SUBTOTAL**, **TAX**, and **CASH TENDERED** with the final **TOTAL**, or attempt to recompute totals, introducing arithmetic hallucinations.
- Taxonomy & Category Drift:** Open-ended prompting can yield unconstrained categories (e.g., "Pantry", "Outings", "Snacks"), breaking downstream schemas and relational integrity.
- Missing Field Handling:** Receipts frequently omit dates or merchant headers due to thermal wear, so the system must handle missing values explicitly rather than hallucinating.

## 3. Methods & Architectural Paradigms Compared



Pipelines Compared (Generated by AI and verified)

### 3.1 Pipeline A: Two-Stage OCR → Text LLM Structuring

- **Architecture:** An OCR engine (**PaddleOCR** with text-angle classification and directional bounding-box sorting) transcribes the image into a 1D text stream. This intermediate text is then passed to a text-only LLM for named entity recognition (NER) and JSON structuring.
- **Failure Modes:**
  - *Compounding Error Propagation:* If OCR misreads `$14.99` as `$14.09` or drops a decimal point, the downstream LLM cannot recover the ground truth.
  - *Loss of 2D Spatial Geometry:* Flattening multi-column layouts into line-by-line text can disconnect item descriptions from their right-aligned prices.

### 3.2 Pipeline B: End-to-End Multimodal VLM (Production Architecture)

- **Architecture:** Raw image bytes (Base64/JPEG) are passed directly into a multimodal transformer alongside an engineered system prompt and a strict `response_schema`.
- **Advantages:**
  - *Joint Layout & Visual Semantics:* Vision-language attention layers can jointly attend to visual cues (logos, bold fonts, alignment) and textual content.
  - *Schema-Constrained Decoding:* The model's token space is restricted using a context-free grammar (CFG) or constrained logits, ensuring syntactically valid JSON and preventing parser failures.

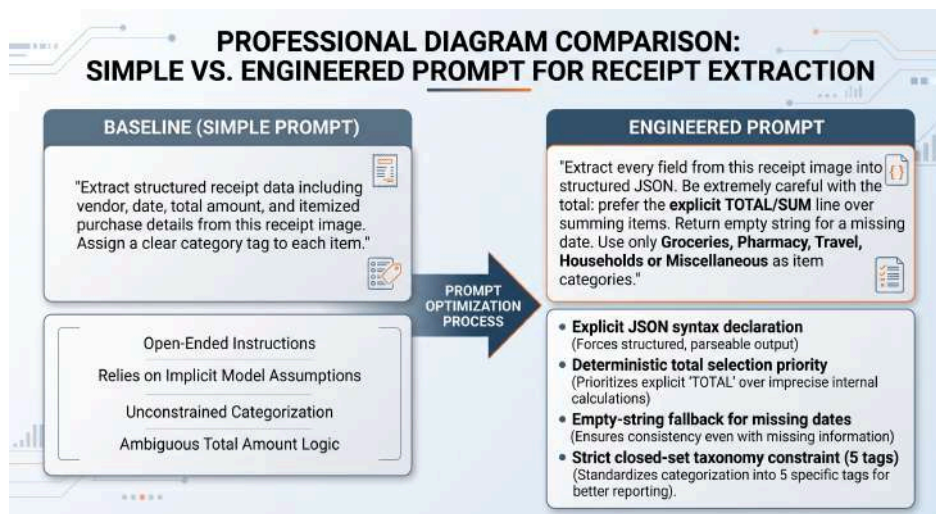
### 3.3 Hybrid Fallback: Multinomial Naive Bayes & Cascade Classifier

For manual expense entries or unmapped granular labels, the client architecture integrates an on-device, dependency-free **Multinomial Naive Bayes classifier** operating in log-space with Laplace smoothing.

A deterministic cascade executes with explicit confidence prioritization: **User Selection (1.00)** → **Merchant Exact Match (0.95)** → **Keyword Match (0.85)** → **Receipt Sub-Category Evidence (0.80)** → **Naive Bayes Inference (0.70)** → **Fallback (0.10)**.

## 4. Prompt Engineering: Baseline vs. Engineered Formulation

The main NLP improvement in the receipt extraction engine comes from systematically refining the system prompt to remove ambiguity and enforce deterministic structure.



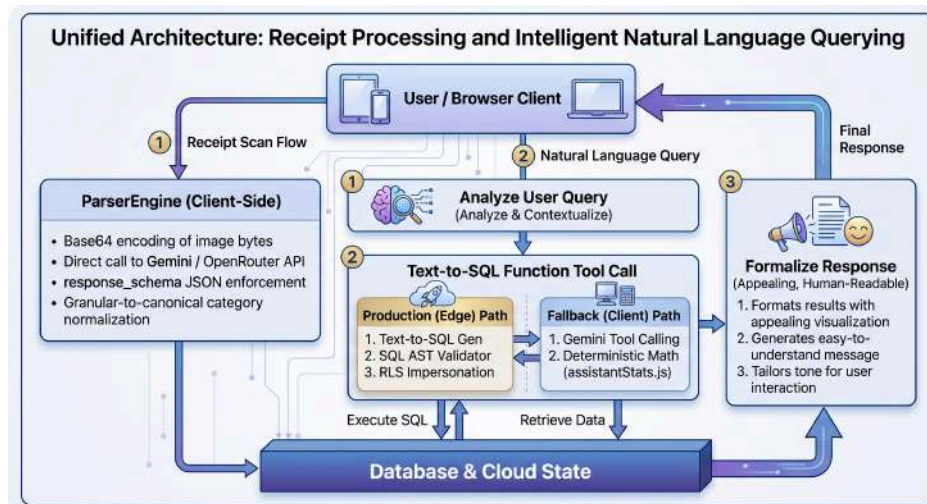
Prompt Engineering of the parser prompt (Generated by AI and verified)

### 4.2 Core Prompt Engineering Principles Applied



| Prompt Engineering Technique                  | Simple Prompt Limitation                                                                                           | Engineered Prompt Solution                                                                                           | NLP / LLM Mechanism                                                                                                                          |
|-----------------------------------------------|--------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| <b>1. Closed-Vocabulary Constraint</b>        | Used open phrase: <i>"Assign a clear category tag"</i> , leading to taxonomy explosion.                            | Constrained: <i>"Use only Groceries, Pharmacy, Travel, Households or Miscellaneous"</i>                              | Shrinks the categorical output token space and aligns outputs with canonical database enums.                                                 |
| <b>2. Arithmetic Disambiguation Directive</b> | Models often summed line items, creating errors when taxes, tips, or discounts were present.                       | Specified rule: <i>"Be extremely careful with the total: prefer the explicit TOTAL/SUM line over summing items."</i> | Discourages internal recalculation and prioritizes extraction from visual anchor keywords ( <b>TOTAL</b> , <b>SUM</b> , <b>AMOUNT DUE</b> ). |
| <b>3. Deterministic Null Handling</b>         | Missing dates caused nulls, omitted keys, or hallucinated timestamps.                                              | Explicit instruction: <i>"Return empty string for a missing date."</i>                                               | Provides a deterministic fallback ( <b>" "</b> ) that prevents schema and parser exceptions.                                                 |
| <b>4. Schema Adherence Instruction</b>        | Requested generic <i>"structured receipt data"</i> , which could be returned as Markdown, YAML, or arbitrary text. | Demanded <i>"structured JSON"</i> alongside schema constraints.                                                      | Primes generation toward standard JSON syntax tokens ( <b>{</b> , <b>"</b> , <b>:</b> ).                                                     |

## 5. System Architecture & AI Assistant Execution



Unified System Architecture (Generated by AI and verified)

### 5.1 Text-to-SQL Edge Function with Read-Only RLS Impersonation

SnapSpend implements a Text-to-SQL natural language interface as a Supabase Deno Edge Function:

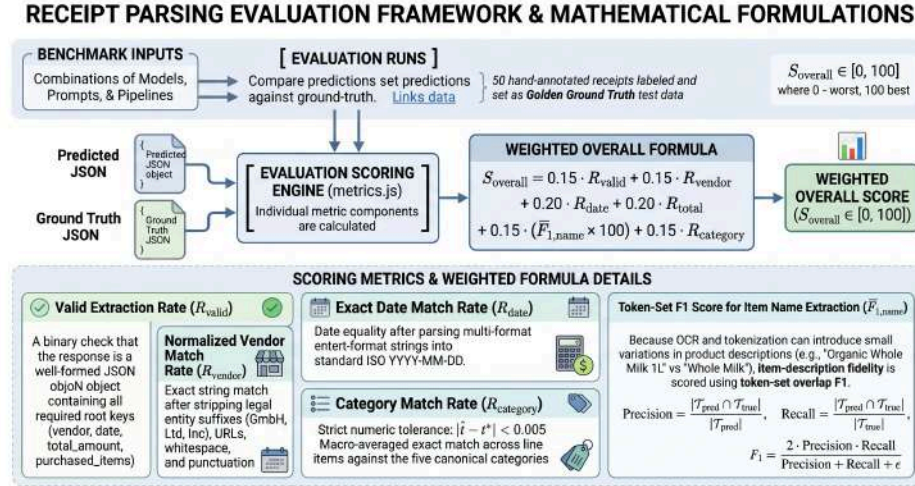
- JWT Verification:** Authenticates the user identity via JWKS.
- Constrained SQL Generation:** The LLM generates a single SQL query against the financial schema.
- AST & Syntax Validation Gate:** Disallows mutations, multiple statements, subqueries, **UNION** , or unapproved tables.
- RLS Impersonation:** Executes within a read-only transaction under **SET LOCAL ROLE authenticated** , ensuring the database enforces row-level privacy even if the model produces an unconstrained **WHERE** clause.

### 5.2 Deterministic Client-Side Tool-Calling Fallback

For serverless/static deployments, the assistant uses function calling against eight deterministic aggregators (e.g., **category\_breakdown** , **financial\_summary** , **month\_over\_month** ) implemented in pure JavaScript. This avoids arithmetic hallucination by delegating computation to deterministic functions.

## 6. Evaluation Framework & Mathematical Formulations

To benchmark model and prompt variations, an evaluation runs combinations of models, prompts, and pipelines against human-annotated, ground-truth receipts, we collected **50 receipts** and labeled them accurately and set their corresponding **JSON** parsed output as the **golden ground truth** test data, and then compared the results of multiple settings against the ground truth using the following diagram:



Evaluation metrics and full pipeline (Generated by AI and verified)

## 6.1 Scoring Metrics & Weighted Formula

We designed the benchmark as a composite overall score ( $S_{\text{overall}} \in [0, 100]$ ) weighted across six core dimensions:

$$S_{\text{overall}} = 0.15 \cdot R_{\text{valid}} + 0.15 \cdot R_{\text{vendor}} + 0.20 \cdot R_{\text{date}} + 0.20 \cdot R_{\text{total}} + 0.15 \cdot (\bar{F}_{1,\text{name}} \times 100) + 0.15 \cdot R_{\text{category}}$$

- Valid Extraction Rate ( $R_{\text{valid}}$ ):** A binary check that the response is a well-formed JSON object containing all required root keys (vendor, date, total\_amount, purchased\_items).
- Normalized Vendor Match Rate ( $R_{\text{vendor}}$ ):** Exact string match after stripping legal entity suffixes (GmbH, Ltd, Inc), URLs, whitespace, and punctuation.
- Exact Date Match Rate ( $R_{\text{date}}$ ):** Date equality after parsing multi-format strings into standard ISO YYYY-MM-DD.
- Total Amount Exact Rate ( $R_{\text{total}}$ ):** Strict numeric tolerance:  $|\hat{t} - t^*| < 0.005$ .
- Category Match Rate ( $R_{\text{category}}$ ):** Macro-averaged exact match across line items against the five canonical categories.

## 6.2 Token-Set F1 Score for Item Name Extraction ( $\bar{F}_{1,\text{name}}$ )

Because OCR and tokenization can introduce small variations in product descriptions (e.g., "Organic Whole Milk 1L" vs "Whole Milk"), item-description fidelity is scored using token-set overlap F1:

$$\text{Precision} = \frac{|\mathcal{T}_{\text{pred}} \cap \mathcal{T}_{\text{true}}|}{|\mathcal{T}_{\text{pred}}|}, \quad \text{Recall} = \frac{|\mathcal{T}_{\text{pred}} \cap \mathcal{T}_{\text{true}}|}{|\mathcal{T}_{\text{true}}|}, \quad F_1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall} + \epsilon}$$

## 7. Empirical Results & Comparative Analysis

The evaluation was run across five multimodal architectures and one OCR pipeline using the **Engineered Prompt**.

(Due to **Quota limitation** on free versions we couldn't replicate the **simple prompt** results but it was slightly worse than the engineered versions as mentioned)

### 7.1 Quantitative Benchmark Results

| model                       | overall<br>( $S_{\text{overall}}$ ) | valid rate<br>( $R_{\text{valid}}$ ) | vendor rate<br>( $R_{\text{vendor}}$ ) | date rate<br>( $R_{\text{date}}$ ) | total exact rate<br>( $R_{\text{total}}$ ) | item count rate | name<br>( $\bar{F}_{1,\text{name}}$ ) |
|-----------------------------|-------------------------------------|--------------------------------------|----------------------------------------|------------------------------------|--------------------------------------------|-----------------|---------------------------------------|
| gemini-3.1-flash-lite       | 95.97                               | 100.00                               | 87.50                                  | 100.00                             | 100.00                                     | <b>97.50</b>    | 0.9656                                |
| gemma-4-31b-it              | 93.03                               | 100.00                               | 75.00                                  | 92.50                              | 100.00                                     | <b>92.50</b>    | 0.9601                                |
| gemma-4-26b-a4b-it          | 93.40                               | 100.00                               | 85.00                                  | 95.00                              | 100.00                                     | <b>95.00</b>    | 0.9381                                |
| gemini-3.7-flash            | 100.00                              | 100.00                               | 100.00                                 | 100.00                             | 100.00                                     | <b>100.00</b>   | 1.0000                                |
| gemini-3.1-flash-lite (ocr) | 86.46                               | 100.00                               | 58.00                                  | 100.00                             | 100.00                                     | <b>86.00</b>    | 0.7572                                |

## 7.2 In-Depth Performance & Error Analysis

### 1. Total Amount Reliability (100% Across All Models):

The engineered instruction (*"prefer the explicit TOTAL/SUM line over summing items"*) eliminated arithmetic hallucinations and subtotal confusion across all five models ( $R_{\text{total}} = 100.0\%$ , Total Rel. Err = 0.0000).

### 2. Taxonomy & Category Conformance:

Restricting categories to the five canonical classes produced strong categorization accuracy. Frontier models ( `gemini-3.7-flash` ) achieved **100% category match**, while smaller/quantized models ( `gemma-4-26b-a4b-it` ) showed a modest drop (**83.88%**) and occasionally drifted on ambiguous items (e.g., household toiletries vs. pharmacy items), and the absolute lowest rate happened with `gemini-3.1-flash-lite ( ocr )` (**76%**) due to mistakes in the spelling of items in the OCR which resulted in wrong classification of categories of those items.

### 3. Vendor Header Disambiguation:

Open-weight models showed lower vendor-identification performance ( `75.0%` for `gemma-4-31b-it` and `85.0%` for `gemma-4-26b-a4b-it` ), occasionally confusing franchise locations or slogans with the primary vendor name, but on the other side, the **OCR** pipeline had the worst capture on the vendor name to only (**58%**) of the time to get it correctly.

### 4. Zero Structural Parse Errors:

All models achieved a **100% valid rate** with zero errors, validating the effectiveness of structured JSON prompting combined with robust downstream JSON recovery.

## 8. Our AI Usage in building *SnapSpend*

We learnt about **OpenCode** and and learnt in a short window of time how to use it to **vibe code** many parts of the project especially the **user interface** —**And it is very very nice!**, also **Gemini-Flash-3.7** was used **with the right context** to create the correct **diagrams** for us -after **verifying** its outputs-.

## 9. Conclusion

This report evaluated the transition from cascaded OCR pipelines to end-to-end Multimodal Large Language Models for receipt document intelligence. The results indicate that:

1. End-to-end multimodal perception mitigates the compounding errors inherent in OCR-to-NLP pipelines.
2. Targeted prompt engineering—especially closed-set category constraints, explicit arithmetic anchor selection, and deterministic null handling—reduces common structured-extraction failure modes.
3. Frontier multimodal model ( `gemini-3.7-flash` ) achieved 100% extraction accuracy across all evaluation dimensions under structured prompting constraints, demonstrating the viability of zero-shot vision-language architectures for enterprise financial document understanding.